

StudyVerse

E-Notes Sharing Application

"Empowering Minds, Sharing Knowledge"

DETAILED PROJECT REPORT

Project Title: StudyVerse - E-Notes Sharing Application

Developed By: Ayush Kumar

University: Lovely Professional University (LPU)

Course: B.Tech Computer Science Engineering

Semester: 7th Semester

AWS EC2 URL: <http://23.20.190.164>

AWS RDS Host: dbstudyverse.c6nwkq8cuodu.us-east-1.rds.amazonaws.com

AWS S3 Bucket: [studyverse-uploads \(us-east-1\)](#)

GitHub Repo: github.com/Ayush12708/e-notes-sharing-application

Date: July 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Technology Stack
5. System Architecture
6. Database Design (ER Diagram & Schema)
7. Module Description
8. Functional Requirements
9. Non-Functional Requirements
10. Implementation Details
11. URL Routing Structure
12. Screenshots & User Interface
13. Testing & Validation
14. Future Scope
15. Conclusion

1. Introduction

StudyVerse is a full-stack web-based E-Notes Sharing Application designed to allow college and university students to upload, share, browse, and manage academic study materials such as lecture notes, PDF documents, and digitally-created e-notes. The platform enables peer-to-peer knowledge exchange across multiple engineering branches and semesters with features like bookmarking, commenting, likes, admin content moderation, and real-time view tracking.

The application follows the Django MVT (Model-View-Template) architectural pattern and is deployed live on AWS EC2 (Ubuntu 24.04, Nginx, Gunicorn) backed by an AWS RDS MySQL 8.0 production database instance.

Tagline: "Empowering Minds, Sharing Knowledge"

2. Problem Statement

College students often face significant challenges in accessing quality study materials:

- **Fragmented Resources:** Notes are scattered across WhatsApp groups, Google Drive links, and personal devices with no centralized access point.
- **No Quality Control:** Shared materials often lack verification, leading to incorrect or incomplete content.
- **Limited Discovery:** Students struggle to find notes for specific subjects, semesters, or branches.
- **No Collaboration:** Traditional file sharing provides no mechanism for discussions or feedback.
- **Loss of Materials:** Students frequently lose access when WhatsApp groups are deleted or Drive links expire.

StudyVerse solves these problems by providing a permanent, centralized, admin-verified platform where students can upload, discover, bookmark, and discuss study materials with real-time analytics.

3. Objectives

1. Build a centralized platform where students from any branch can upload and access notes.
2. Support file uploads (PDF, DOCX, PPTX, images) and online rich-text e-note creation with whiteboard drawing canvas.
3. Implement admin content moderation workflow (Pending -> Approved / Rejected).
4. Enable smart search and filtering by subject, branch, semester, keyword, and sorting.
5. Allow users to like, bookmark, and comment on notes for academic discussion.
6. Track real-time note view counts and display live statistics.
7. Provide secure multi-user authentication with concurrent session handling.
8. Deploy cloud infrastructure on AWS EC2 with AWS RDS MySQL Database.

4. Technology Stack

Layer	Technology	Purpose
Backend	Django 6.0 (Python 3.12)	Server-side logic, ORM, auth
Cloud Server	AWS EC2 (Ubuntu 24.04)	Virtual machine server
Cloud Database	AWS RDS MySQL 8.0	Production relational database
Web Server	Nginx 1.24 + Gunicorn	Reverse proxy & WSGI daemon
Frontend	HTML5, CSS3, JavaScript	Templates, single-card UI design
CSS	Custom Design System	1300+ lines hand-crafted CSS
Serverless	Vercel Serverless	Cloud preview deployment
Deployment Script	deploy_ec2.sh	1-command automated AWS setup

Python Dependencies

requirements.txt: django, gunicorn, whitenoise, pymysql, fpdf2

5. System Architecture

5.1 Django MVT Flow

Client Browser -> Nginx Reverse Proxy (Port 80) -> Gunicorn WSGI Daemon -> Django URL Dispatcher -> Views -> Models (ORM queries to AWS RDS MySQL) + Templates -> Rendered Response back to Client

5.2 Project Directory Structure

- config/ - Project settings, URLs, WSGI (AWS RDS & MySQL fallback)
- accounts/ - User auth: models, views, forms, signals, admin
- notes/ - Core CRUD: models, views, whiteboard canvas, moderation
- dashboard/ - Student dashboard: views, URLs, real-time metrics
- home/ - Landing page: views, URLs, department grid
- templates/ - Single-card UI templates (accounts, notes, dashboard)
- deploy_ec2.sh - Automated 1-command AWS EC2 deployment script
- generate_report_pdf.py - Standalone FPDF2 PDF report generator script
- vercel.json - Vercel serverless deployment routing file
- static/css/ - Custom CSS design system (1300+ lines)

- media/notes/ - Uploaded note files (PDF, DOCX, PPTX, images)

6. Database Design

6.1 ER Relationships

auth_user (1) ----> (1) accounts_profile [One-to-One]
auth_user (1) ----> (N) notes_note [One-to-Many]
notes_note (1) ---> (N) notes_bookmark [One-to-Many]
notes_note (1) ---> (N) notes_comment [One-to-Many]
notes_note (N) <--> (N) auth_user via notes_note_likes [Many-to-Many]

6.2 MySQL Table Schema

Table Name	Key Fields	Description
auth_user	id, username, password, email, is_staff	Django built-in user auth table
accounts_profile	user_id (FK), phone, college, semester	Extended profile (1:1 with User)
notes_note	title, subject, branch, semester, content, file, status, views	Core notes table
notes_bookmark	user_id (FK), note_id (FK), created_at	Saved bookmarks
notes_comment	note_id (FK), user_id (FK), text	Discussion comments
notes_note_likes	note_id (FK), user_id (FK)	Many-to-many likes
django_session	session_key, session_data, expire_date	Active login sessions

7. Module Description

Module 1: Accounts (Authentication & Profile)

- **Registration:** Users register with username, email, password, phone, college, semester. Passwords hashed with PBKDF2.
- **Login:** Case-insensitive username matching. Supports redirect after login (?next= parameter).
- **Profile:** View and edit personal details. Auto-created via Django signals for all user creation methods.
- **Admin:** Profile displayed inline in Django Admin User page.

Module 2: Notes (Core CRUD & Content)

- **Upload File Note:** Upload PDF, DOCX, PPTX, images with metadata (title, subject, branch, semester).
- **Create Online E-Note:** Rich-text editor with whiteboard drawing canvas for diagrams.
- **Browse Notes:** Multi-filter search by subject, branch, semester, keyword. Sorting by newest, oldest, A-Z, popular.
- **Note Detail:** Full display with inline PDF/image preview, view counter, likes, bookmarks, comments.
- **Real-Time Views:** Atomic F() expression increments view count on every page load.
- **Edit/Delete:** Authors can modify or remove their own notes.
- **Draft System:** Save notes as Draft without submitting for review.

Module 3: Social Engagement

- **Bookmarks:** Toggle save/unsave on any note. AJAX-powered. Dedicated bookmarks page.
- **Likes:** Toggle like/unlike. AJAX response returns updated count.
- **Comments:** Post discussion comments on note detail page. Ordered by newest.

Module 4: Admin Content Moderation

Staff-only Admin Dashboard showing all notes with Pending/Approved/Rejected counts. Admin can approve or reject any note. 4-state workflow: Draft -> Pending -> Approved / Rejected. Only approved notes visible in Browse.

Module 5: Dashboard (Student Analytics)

Personal stats (total notes, approved, pending, downloads, likes), recent notes, bookmarks, subject exploration with live counts, and community feed.

Module 6: Home (Landing Page)

Hero section with live stats, Engineering Departments grid, About section with 3-step workflow, Trending Notes preview, Features showcase, Stats banner, and 3-column footer.

8. Functional Requirements

ID	Requirement	Status
FR-01	Register with username, email, password, phone, college, semester	Done
FR-02	Login with username and password (case-insensitive)	Done
FR-03	View and update profile information	Done
FR-04	Upload notes as PDF/DOCX/PPTX/Image files	Done
FR-05	Create online e-notes with rich text and whiteboard	Done
FR-06	Browse notes with search, filter, and sort	Done
FR-07	View full note details with inline file preview	Done
FR-08	Real-time view counter on note open	Done
FR-09	Bookmark/unbookmark notes	Done
FR-10	Like/unlike notes	Done
FR-11	Post comments on notes	Done
FR-12	Edit/delete own notes	Done
FR-13	Save notes as Draft	Done
FR-14	Admin dashboard to view all notes	Done
FR-15	Admin approve/reject pending notes	Done
FR-16	All data stored in MySQL via Workbench	Done
FR-17	Auto-create Profile for users created by any method	Done

9. Non-Functional Requirements

Requirement	Implementation
Security	PBKDF2_SHA256 password hashing, CSRF protection, @login_required decorators, @staff_member_required
Performance	Django F() atomic updates, lazy QuerySet, select_related() for JOIN optimization
Scalability	MySQL production DB, concurrent sessions, Gunicorn multi-worker WSGI
Responsiveness	Fully responsive CSS with breakpoints at 992px and 640px
Usability	Clean academic blue palette, flash messages, intuitive navigation

10. Implementation Details

10.1 User Registration Flow

1. User fills registration form (username, email, password, phone, college, semester)
2. Form validated (password match, unique username check)
3. `User.objects.create_user()` creates entry in `auth_user` with hashed password
4. `Profile.objects.create()` creates entry in `accounts_profile`
5. User is automatically logged in via `login(request, user)`
6. Redirected to Dashboard with success message

10.2 Django Signal for Auto-Profile

A `post_save` signal on the User model automatically creates a Profile record whenever a new User is created - whether via website registration, Django admin, `manage.py createsuperuser`, Django shell, or any backend script.

10.3 Real-Time View Counter

Uses Django F() expression: `Note.objects.filter(pk=pk).update(views=F("views") + 1)`. This generates atomic SQL: `UPDATE notes_note SET views = views + 1`, preventing race conditions with concurrent users.

10.4 MySQL Connection

PyMySQL is installed as MySQLdb compatible driver. `settings.py` configures `ENGINE=django.db.backends.mysql` connecting to `notehub_db` on `localhost:3306`.

10.5 Content Moderation Workflow

User uploads note -> Status = "Pending" -> Admin reviews in Dashboard -> Approved (visible in Browse) or Rejected (hidden). Users can also save as "Draft" for later editing.

11. URL Routing Structure

URL Pattern	View	Description
/	home	Landing page
/accounts/register/	register	User registration
/accounts/login/	login_view	User login
/accounts/logout/	logout_view	User logout
/accounts/profile/	profile_view	View/edit profile
/dashboard/	dashboard	Student dashboard
/notes/upload/	upload_note	Upload file note
/notes/create-online/	create_online_note	Create e-note
/notes/browse/	browse_notes	Browse & search notes
/notes/detail/<id>/	note_detail	View note details
/notes/my-notes/	my_notes	User uploaded notes
/notes/edit/<id>/	edit_note	Edit own note
/notes/delete/<id>/	delete_note	Delete own note
/notes/bookmark/<id>/	toggle_bookmark	Save/unsave bookmark
/notes/like/<id>/	toggle_like	Like/unlike note
/notes/bookmarks/	bookmarks_list	View saved bookmarks
/notes/admin-dashboard/	admin_dashboard	Admin moderation
/notes/approve/<id>/	approve_note	Approve a note
/notes/reject/<id>/	reject_note	Reject a note
/admin/	Django Admin	Built-in admin panel

12. Screenshots & User Interface

The UI uses a custom-built CSS design system (1300+ lines) with:

- Color Palette: Academic blue (#2563eb) primary, steel slate neutrals, clean white surfaces
- Typography: Poppins font family from Google Fonts
- Components: Cards, pills, badges, buttons, dropdowns, forms, tables, department grids
- Responsive: Full mobile/tablet/desktop breakpoints at 992px and 640px

Key Pages: Home (hero + departments + about + trending + features + stats + footer), Dashboard, Browse Notes,

Note Detail, Upload Note, Create E-Note, My Notes, Admin Dashboard, Login/Register, Profile.

13. Testing & Validation

Test Case	Expected Result	Status
Register with valid data	User created, logged in, redirected	Pass
Register duplicate username	Error "Username taken"	Pass
Register password mismatch	Error "Passwords do not match"	Pass
Login valid credentials	Redirect to dashboard	Pass
Login invalid credentials	Error message displayed	Pass
Upload PDF note	Saved with Pending status	Pass
Create online e-note	Saved with content + drawing data	Pass
Browse with search filter	Only matching notes shown	Pass
View note detail	View counter increments by 1	Pass
Bookmark/unbookmark	Toggle works correctly	Pass
Like/unlike note	Count updates correctly	Pass
Post comment	Comment appears on detail page	Pass
Admin approve note	Status -> Approved, visible in Browse	Pass
Admin reject note	Status -> Rejected, hidden	Pass
Non-staff access admin panel	403 Forbidden	Pass
Create user from Django shell	Profile auto-created by signal	Pass

14. Future Scope

1. AI-Powered Note Summarization using OpenAI/Gemini API
2. Real-Time Chat via WebSocket study group rooms
3. Email Notifications for note approval/rejection
4. Mobile App using React Native or Flutter
5. OCR Integration to extract text from handwritten notes
6. Gamification with leaderboards, badges, and reward points
7. Multi-Language Support (Hindi and regional languages)
8. Cloud Deployment on AWS/GCP with S3 file storage
9. Advanced Analytics with charts for upload trends and engagement
10. Plagiarism Detection for uploaded content

15. Conclusion

StudyVerse successfully demonstrates a full-stack web application built using the Django framework with MySQL database integration. The platform addresses a real-world problem faced by engineering students - the lack of a centralized, verified, and collaborative study material sharing system.

Key Achievements

- Complete CRUD Operations for notes with file upload and online e-note creation
- Secure Authentication with password hashing, CSRF protection, and session management
- Admin Content Moderation with 4-state workflow (Draft -> Pending -> Approved/Rejected)
- Social Features including bookmarks, likes, and comments
- MySQL Production Database managed via MySQL Workbench
- Responsive UI Design with custom CSS design system (1300+ lines)
- Django Signals for automatic profile creation across all user creation methods
- Real-Time View Tracking using atomic database operations
- Multi-Branch & Semester Filtering for targeted note discovery

The project demonstrates proficiency in Python, Django, MySQL, HTML/CSS/JS, Git, and software engineering principles including MVC/MVT design pattern.

StudyVerse - Empowering Minds, Sharing Knowledge

Report prepared by Ayush Kumar | July 2026